

Reviewer Pack — Minimal Rank of Quandle Structure and DPLL Proof Length

Entry afdafa60d6ac · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-08 12:09:56 UTC

Minimal Rank of Quandle Structure and DPLL Proof Length

Entry ID: afdafa60d6ac **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-08 12:09:56 UTC

1. Conjecture

Field A (mathematical branch): Algebra (Quandles) **Field B** (complexity object): Boolean Satisfiability (DPLL Proof Length)

Statement:

For any given boolean formula in CNF, the minimal rank of its associated quandle structure is linearly correlated with the length of the DPLL proof for that formula, such that the minimal rank $R(\varphi) = \Theta(L(\varphi))$, where $L(\varphi)$ is the length of the DPLL proof.

Rationale (proposer's reasoning):

Quandles are a generalization of groups and may capture non-commutative aspects of boolean operations. If the structure of quandles can effectively encode the proof length, it might provide insights into the complexity of proving satisfiability, which is an unsolved problem in complexity theory.

Taxonomy category: Algebraic_Quandle (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): a6c12886a3cb0465

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if and only if the computed Pearson correlation coefficient r for the data set of quandle ranks ($R(\varphi)$) and DPLL proof lengths ($L(\varphi)$) meets the following conditions: $r \geq 0.7$ AND $p\text{-value} \leq 0.05$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 3 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "minimal rank" AND "quandle structure" AND "DPLL proof length" - "algebraic quandles" AND "Boolean satisfiability problem" AND "proof complexity" - DPLL proof length AND quandle AND CNF boolean formula

Top relevant hits considered: - [http://arxiv.org/abs/cs/0209032v3] Complexity Results on DPLL and Resolution - [http://arxiv.org/abs/1502.02131v2] Extracting verified decision procedures: DPLL and Resolution - [http://arxiv.org/abs/cs/0701184v2] Structure and Problem Hardness: Goal Asymmetry and DPLL Proofs in
 SAT-Based Planning

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=124, elapsed=240.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_cnf(n, m):
        clauses = []
        for _ in range(m):
            clause = [random.randint(1, n), -random.randint(1, n)]
            if random.choice([True, False]):
                clause[0] *= -1
            if random.choice([True, False]):
                clause[1] *= -1
            clauses.append(clause)
        return clauses

    def dpll(cnf):
        def search(assignments):
            if not cnf:
                return True
            literal = next((l for l in range(1, n+1) if l not in assignments and -l not in assignments))
            if literal is None:
                return False
            for value in [True, False]:
                new_assignments = assignments.copy()
                new_assignments[literal] = value
                if search(new_assignments):
                    return True
            return False

        return search(assignments)
```

```

n = max(abs(l) for clause in cnf for l in clause)
return len(cnf), search({})

def quandle_rank(cnf):
    # Simplified mapping of CNF to a quandle rank (placeholder)
    return len(cnf)

n_values = [5, 10, 15, 20, 30, 40]
results = []

for n in n_values:
    for _ in range(5):
        cnf = generate_cnf(n, random.randint(2*n, 3*n))
        rank = quandle_rank(cnf)
        proof_length, _ = dpll(cnf)
        results.append((rank, proof_length))

if not results:
    return {
        "metric_name": "quandle_rank",
        "metric_value": None,
        "instances_tested": 0,
        "n_max": 0,
        "conjecture_holds": False,
        "counterexample": "empty_results"
    }

ranks = [r for r, _ in results]
proof_lengths = [l for _, l in results]

mean_rank = sum(ranks) / len(ranks)
mean_length = sum(proof_lengths) / len(proof_lengths)

covariance = sum((r - mean_rank) * (l - mean_length) for r, l in results)
variance_ranks = sum((r - mean_rank)**2 for r in ranks)
variance_lengths = sum((l - mean_length)**2 for l in proof_lengths)

if variance_ranks == 0 or variance_lengths == 0:
    return {
        "metric_name": "quandle_rank",
        "metric_value": None,
        "instances_tested": len(results),
        "n_max": max(n_values),
        "conjecture_holds": False,
        "counterexample": "zero_variance"
    }

correlation_coefficient = covariance / (math.sqrt(variance_ranks) * math.sqrt(variance_lengths))

return {
    "metric_name": "quandle_rank",
    "metric_value": correlation_coefficient,
    "instances_tested": len(results),
    "n_max": max(n_values),
    "conjecture_holds": abs(correlation_coefficient) >= 0.7 and p_value <= 0.05,
    "counterexample": ""
}

```

```

}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    if not results:
        print("RESULT: INCONCLUSIVE reason=empty_results")
    else:
        mean_value = sum(r["metric_value"] for r in results) / len(results)
        std_value = math.sqrt(sum((r["metric_value"] - mean_value)**2 for r in results) / len(results))
        support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

        if support_fraction >= 0.8:
            print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
        elif any(not r["conjecture_holds"] for r in results):
            first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
            print(f"RESULT: FALSIFIED counterexample=\"\" first_failing_seed={first_failing_seed}")
        else:
            print("RESULT: INCONCLUSIVE reason=insufficient_support")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test timed out before producing data, which means the Pearson correlation coefficient and p-value could not be computed to verify the conjecture. | next: NONE

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	16025
2	preregistration	ollama_remote	glm4:latest	0	0	10417
3	novelty	ollama_remote	glm4:latest	0	0	8436
4	novelty	ollama_remote	glm4:latest	0	0	10903
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	23225
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10142
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13445
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12176
9	judge	ollama_remote	glm4:latest	0	0	12982

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 117751 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/afda60d6ac.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/afda60d6ac.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/afda60d6ac.tar.gz` (if generated)